



Name : Farukh Hasan Shaha

Div: SY - B

Roll No. : B - 35

Branch: AI&DS

Aim :

Numerical Computation with NumPy

Theory :

1.Explain the role of NumPy in the Python scientific computing ecosystem.

ANS : NumPy is the core library for numerical computing in Python. It provides powerful N-dimensional arrays and functions to perform fast mathematical and statistical operations. It acts as the foundation for many scientific libraries.

2.Why is NumPy considered faster than pure Python computations?

ANS : It is implemented in C language (low-level, optimized) Uses vectorized operations (no need for loops) Stores data in continuous memory blocks, making computations efficient

3.What types of mathematical operations can be performed using NumPy?

ANS : NumPy supports: Arithmetic operations (addition, subtraction, multiplication, division) Statistical operations (mean, median, standard deviation) Linear algebra (matrix multiplication, determinants) Trigonometric functions (sin, cos, tan) Logarithmic and exponential functions

4.Explain indexing and slicing in NumPy arrays.

ANS: Indexing: Access specific elements using position Example: `arr[0]` (first element) Slicing: Access a range of elements Example: `arr[1:4]` (elements from index 1 to 3) It can also handle multi-dimensional indexing like `arr[0,1]`.

5.How does NumPy integrate with libraries like: a. Pandas b. Matplotlib c. Scikit-learn

ANS: a. Pandas Pandas uses NumPy arrays as its base for DataFrames and Series, making data manipulation faster. b. Matplotlib Matplotlib uses NumPy arrays to plot

graphs and visualize numerical data. c. Scikit-learn Scikit-learn uses NumPy arrays for handling datasets and performing machine learning computations.

```
In [1]: import numpy as np

# 1. Create a NumPy array from a list
scores = [45, 67, 89, 56, 72]
arr = np.array(scores)
print("NumPy Array:", arr)

# 2. Perform basic arithmetic operations
print("Add 5:", arr + 5)
print("Multiply by 2:", arr * 2)

# 3. Find sum and mean
print("Sum:", np.sum(arr))
print("Mean:", np.mean(arr))

# Standard Deviation
print("Standard Deviation:", np.std(arr))

# 4. Reshape a one dimensional array
reshaped_arr = arr.reshape(5, 1)
print("Reshaped Array:\n", reshaped_arr)

# 5. Access elements using indexing
print("First Element:", arr[0])
print("Last Element:", arr[-1])

# Normalization (Min-Max Scaling)
normalized = (arr - np.min(arr)) / (np.max(arr) - np.min(arr))
print("Normalized Scores:", normalized)
```

NumPy Array: [45 67 89 56 72]

Add 5: [50 72 94 61 77]

Multiply by 2: [90 134 178 112 144]

Sum: 329

Mean: 65.8

Standard Deviation: 14.878171930717832

Reshaped Array:

[[45]

[67]

[89]

[56]

[72]]

First Element: 45

Last Element: 72

Normalized Scores: [0. 0.5 1. 0.25 0.61363636]

Conclusion: In this experiment, we used NumPy to perform numerical computations on student scores. We created arrays, applied arithmetic operations, calculated statistical measures like mean and standard deviation, reshaped arrays, and

accessed elements using indexing. We also normalized the data, which helps in comparing scores on a common scale. This experiment shows how NumPy makes numerical analysis fast and efficient.